Desenvolvimento de Microserviços

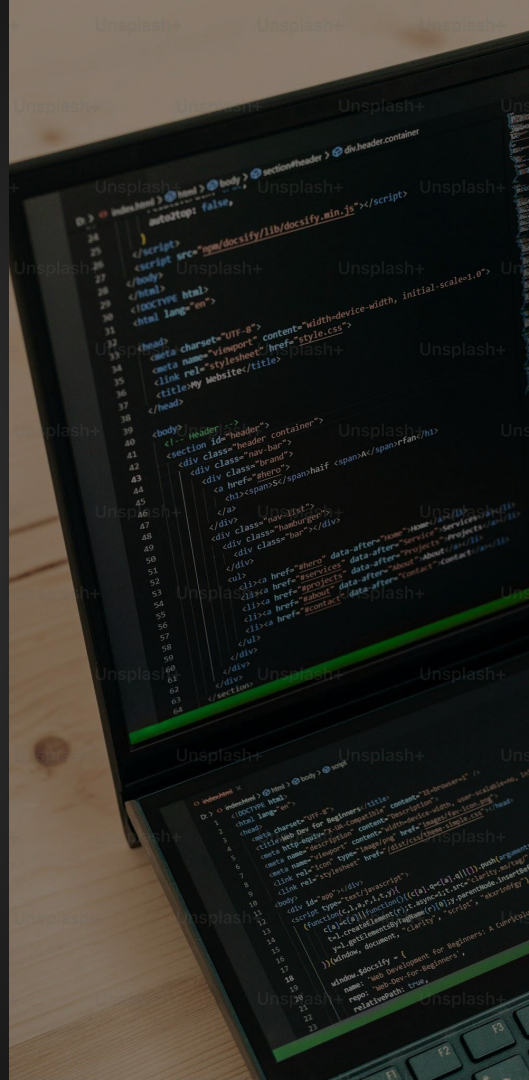
Prof. Guilherme Alves

FACULDADE Donaduzzi

Definição de microsserviço

Um microsserviço é um estilo de arquitetura de software onde um sistema é dividido em pequenos serviços independentes, cada um responsável por uma funcionalidade específica.

- Autonomia
- Especialização
- Escalabilidade
- Comunicação
- Independência Tecnológica

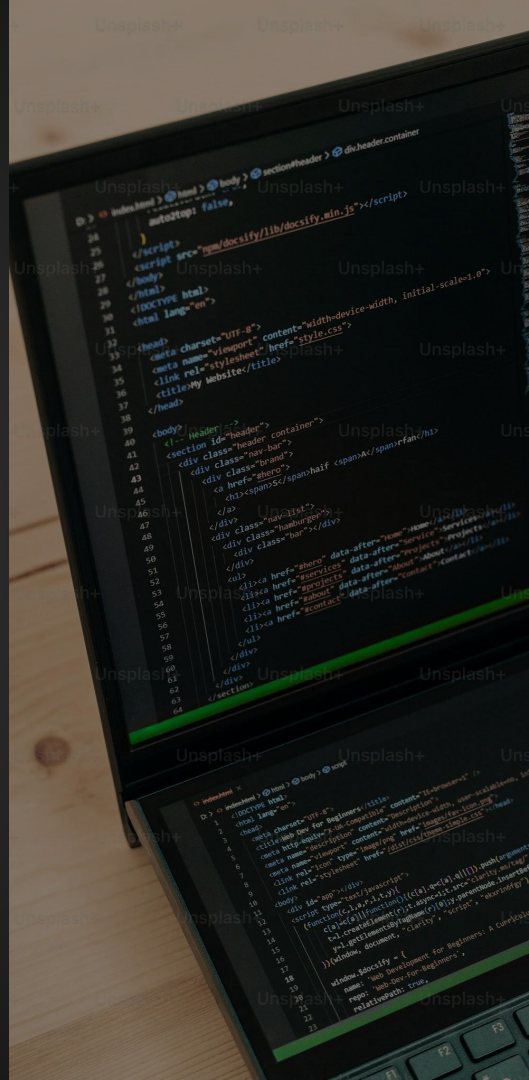


Desafios na comunicação entre microsserviços

A comunicação entre microsserviços traz muitos benefícios, mas também apresenta desafios que devem ser considerados para garantir eficiência, confiabilidade e escalabilidade.

Desafios

- **Latência e performance** - Em micro-serviços as chamadas geralmente passam por redes, aumentando o tempo de resposta.
- **Consistência de dados** - Manter a consistência dos dados entre múltiplos serviços pode ser complexo. Se um serviço falha no meio de uma operação, pode deixar os dados inconsistentes.
- **Gestão de erros e Tolerância a falhas** - Falhas em um serviço podem afetar outros serviços dependentes.
- **Segurança na comunicação** - Cada interação entre serviços pode ser um ponto de vulnerabilidade.



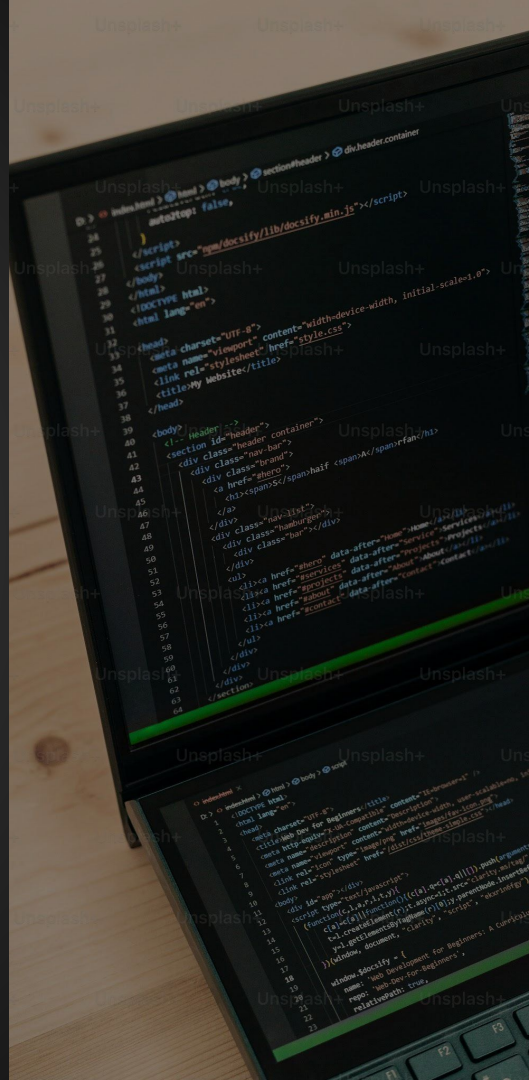
Comunicação

Os microsserviços precisam se comunicar entre si por várias razões essenciais para o funcionamento de sistemas distribuídos. Como cada microsserviço é projetado para ser independente e focado em um único conjunto de funcionalidades, a comunicação entre eles permite que o sistema como um todo funcione de forma coesa e eficiente.

Comunicação síncrona

Na comunicação **síncrona**, o serviço que faz a chamada aguarda uma resposta antes de continuar o processamento.

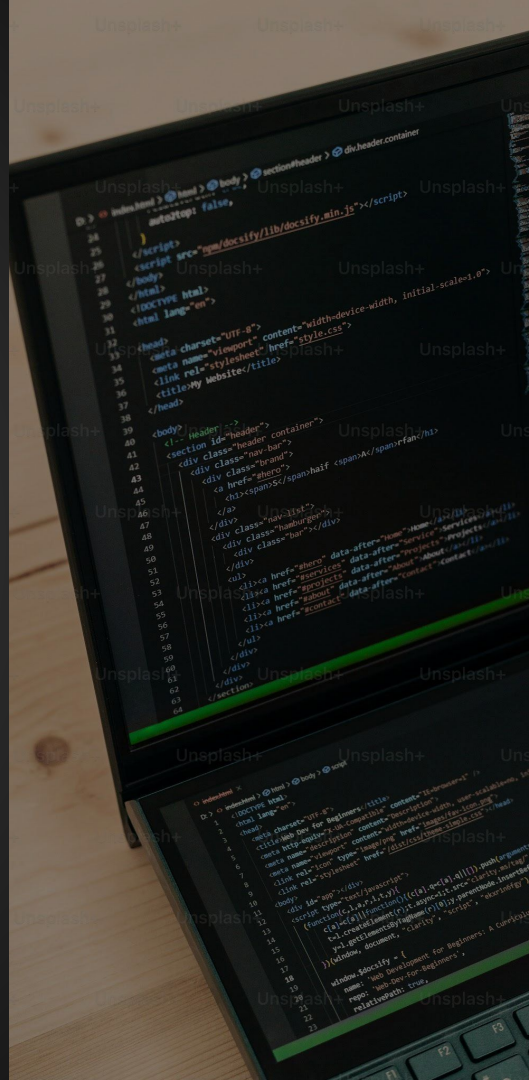
- HTTP
- gRPC
- GraphQL
- WebSockets



Comunicação síncrona

Pode causar maior acoplamento e impacto no desempenho se um serviço estiver indisponível. Porém, em alguns processos a comunicação síncrona é essencial.

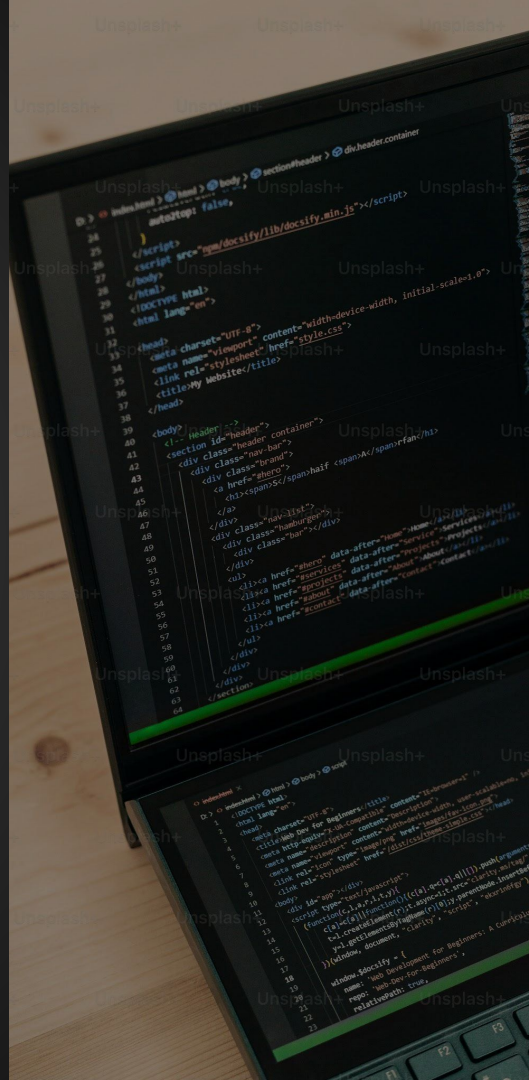
A comunicação síncrona é útil para situações que exigem alta consistência, controle e simplicidade, mas pode ser um desafio quando se trata de escalabilidade e resiliência.



gRPC

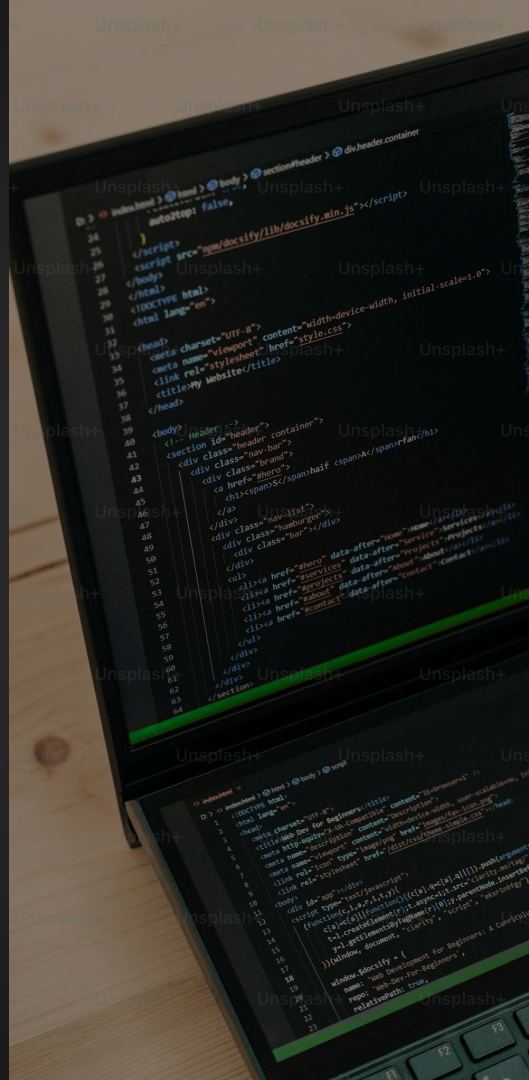
gRPC (*GoogleRemote Procedure Call*) é um framework de comunicação open-source desenvolvido pelo **Google**, que facilita a comunicação eficiente entre microsserviços e aplicações distribuídas.

Ele permite que os serviços se comuniquem de maneira síncrona, por meio de chamadas de procedimentos remotos (*RPC*), utilizando **Protobuf** (*Protocol Buffers*) como formato de serialização de dados.



gRPC - Quando?

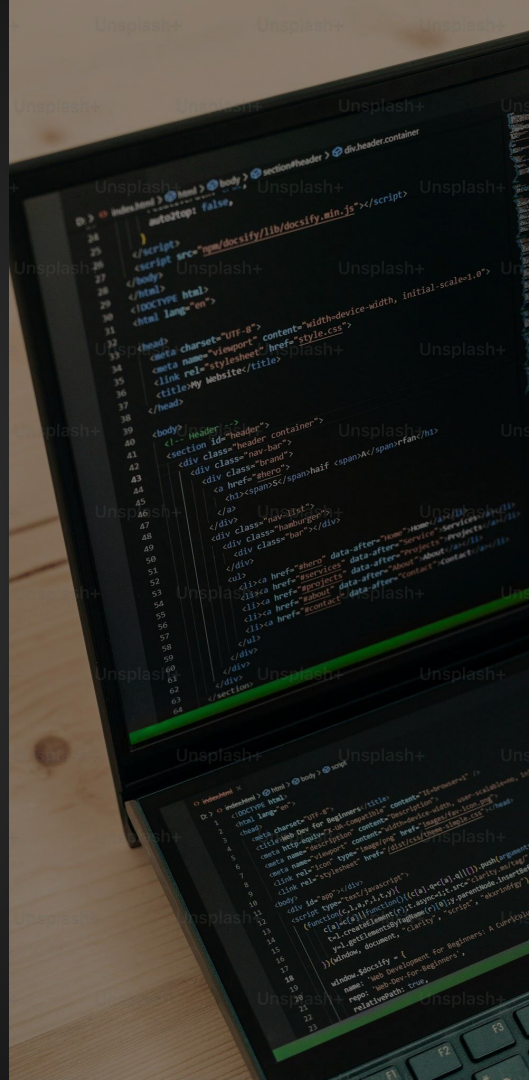
- Quando a performance é um fator decisivo, especialmente em sistemas de comunicação entre microsserviços de baixa latência.
- Quando você precisa de streaming de dados bidirecional ou comunicação contínua, com suporte a multiplexação.
- Quando a comunicação precisa ser eficiente em termos de uso de rede, utilizando binário compacto.
- Quando você está desenvolvendo sistemas em larga escala com muitas interações entre microsserviços.



GraphQL

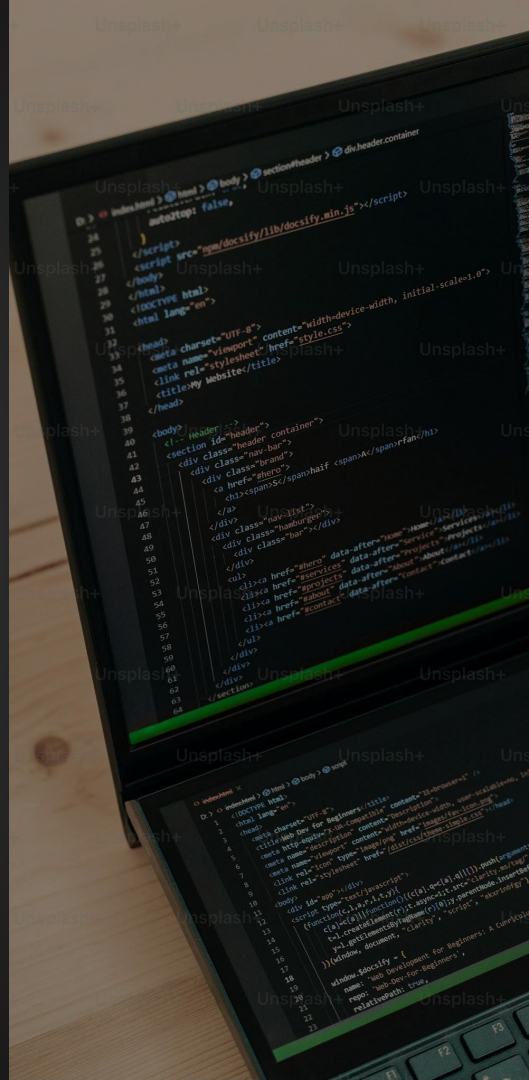
GraphQL é uma linguagem de consulta para APIs, desenvolvida pelo **Facebook**, que permite que o cliente defina exatamente quais dados ele deseja obter de um servidor.

Ao contrário das APIs **REST** tradicionais, onde o servidor define os dados a serem retornados, no **GraphQL** o cliente tem controle total sobre os dados que recebe, o que pode resultar em uma comunicação mais eficiente e flexível entre cliente e servidor.



GraphQL - Quando?

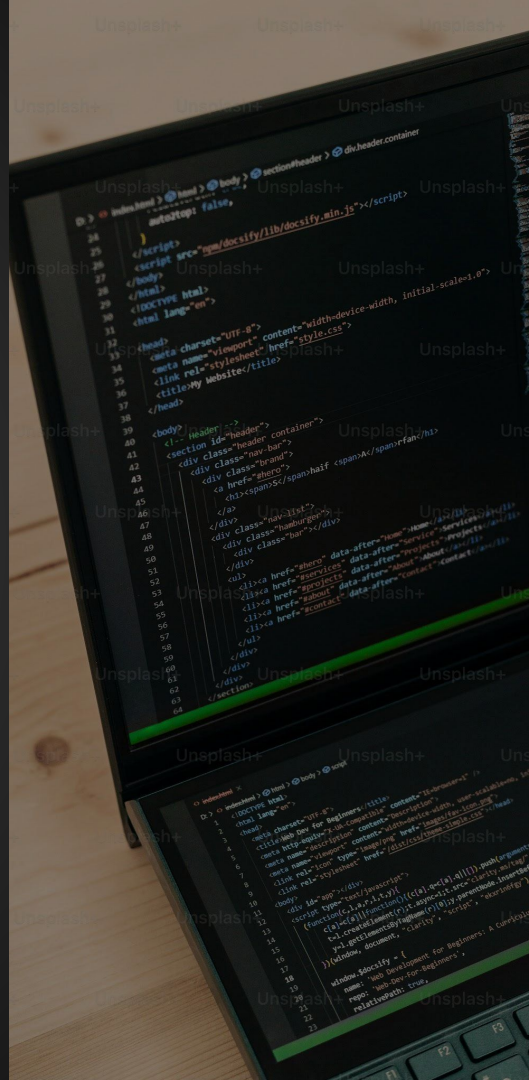
- Quando você precisa de uma API flexível e otimizada para clientes, especialmente em cenários em que o cliente tem controle sobre os dados que deseja receber.
- Quando a aplicação exige diversas fontes de dados e você precisa combinar resultados de diferentes serviços em uma única resposta.
- Quando a comunicação entre serviços deve ser flexível e eficiente, sem realizar múltiplas chamadas para diferentes endpoints.



WebSockets

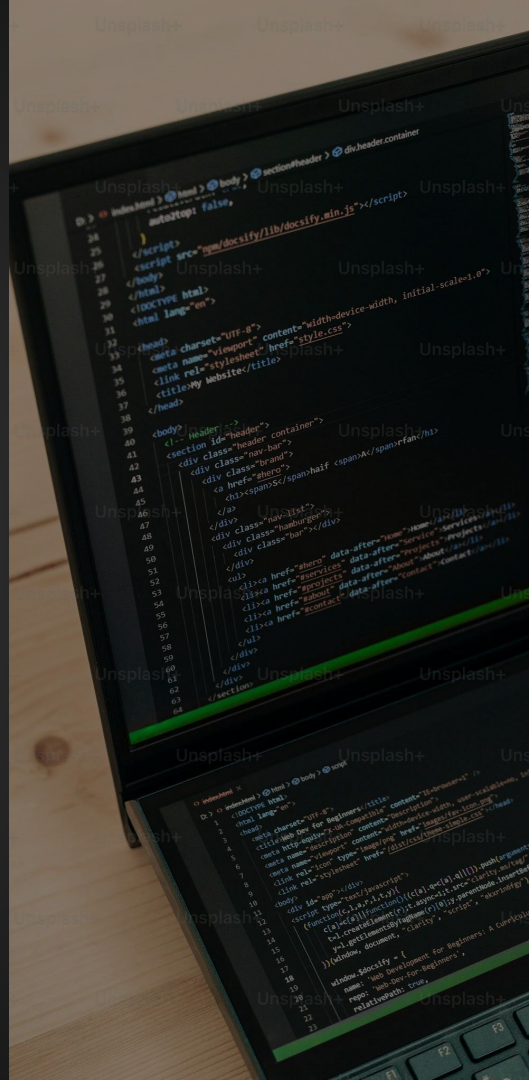
WebSockets é uma tecnologia de comunicação bidirecional em tempo real entre clientes e servidores, permitindo que ambos possam enviar e receber dados sem precisar abrir uma nova conexão a cada mensagem.

Diferente das requisições tradicionais **HTTP**, onde o cliente sempre inicia a comunicação, a conexão permanece aberta, permitindo comunicação contínua e em tempo real.



WebSockets - Quando?

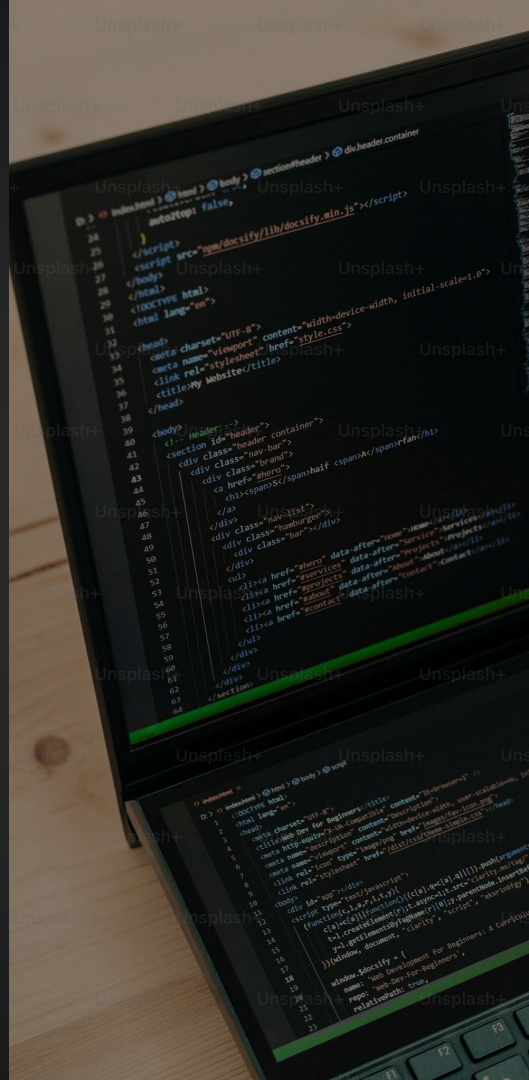
- Quando você precisa de comunicação bidirecional e persistente entre cliente e servidor, sem a sobrecarga de abrir uma nova conexão a cada interação.
- Quando a aplicação exige envio de dados em tempo real, como no caso de jogos, bate-papos, ou sistemas de monitoramento.
- Quando a aplicação não pode esperar por latência ou interrupções, e a comunicação contínua é essencial.



REST

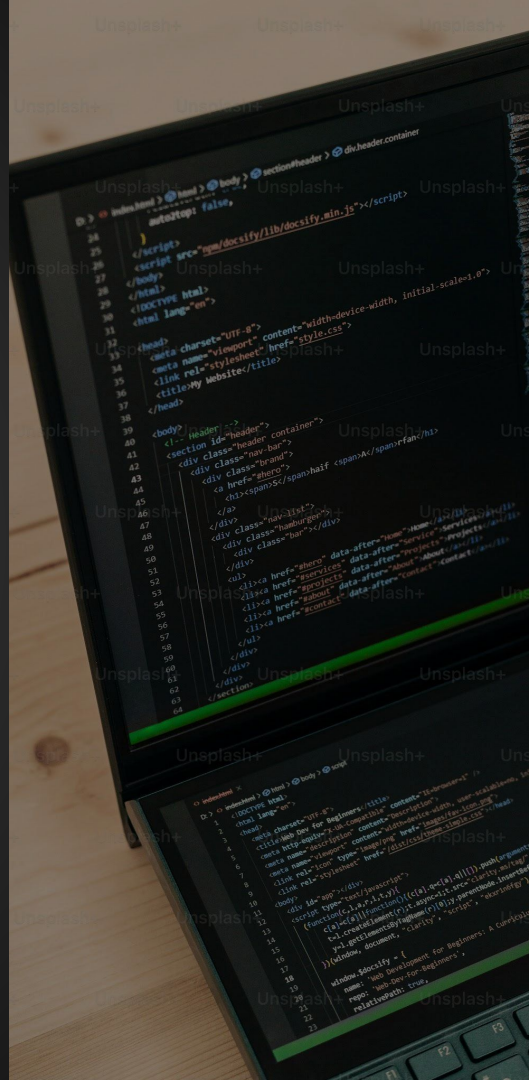
REST (*Representational State Transfer*) é um estilo de arquitetura para a criação de serviços web que se baseia em protocolos **HTTP** para a comunicação entre clientes e servidores.

Seguem uma abordagem mais tradicional de requisição-resposta, onde o cliente envia uma solicitação e o servidor responde com os dados requisitados.



REST - Quando?

- Quando você precisa expor serviços de forma padronizada e simples.
- Quando a simplicidade e a interoperabilidade entre sistemas são mais importantes do que a performance.
- Quando você precisa de um padrão amplamente suportado e fácil de implementar, com boa documentação.
- Quando a comunicação entre microsserviços é simples e não exige comunicação em tempo real ou streaming.
- Quando precisa expor o serviço externamente ao cluster.

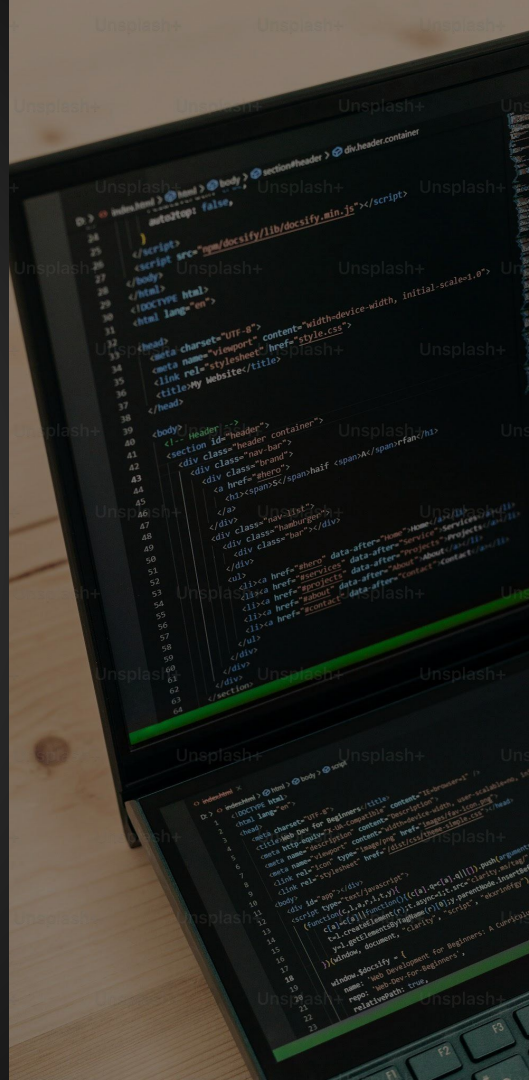


Comunicação assíncrona

Na comunicação assíncrona, a chamada não bloqueia o fluxo do sistema.

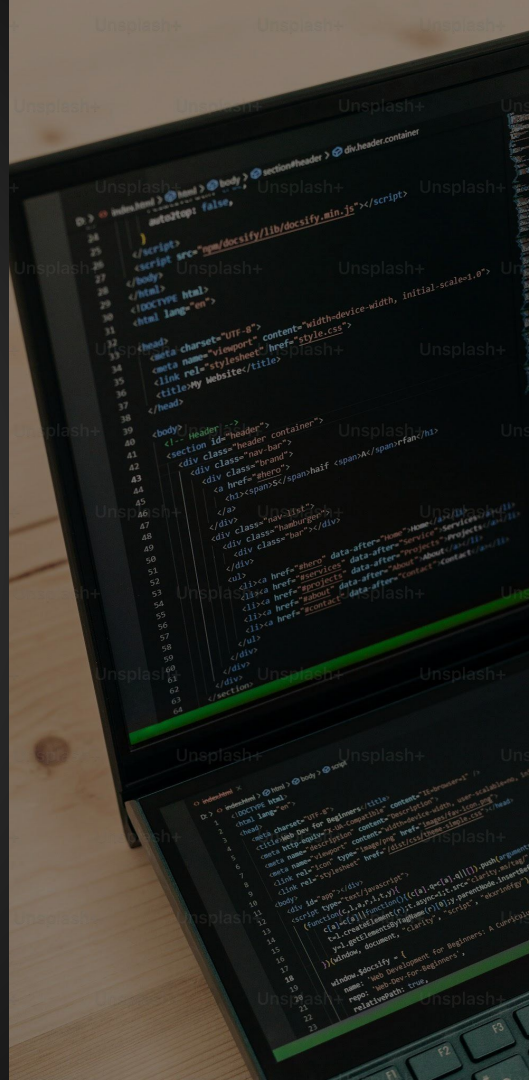
O serviço que envia a mensagem não precisa esperar a resposta imediatamente, permitindo maior escalabilidade e resiliência.

- Mensageria
- Webhook



Comunicação assíncrona

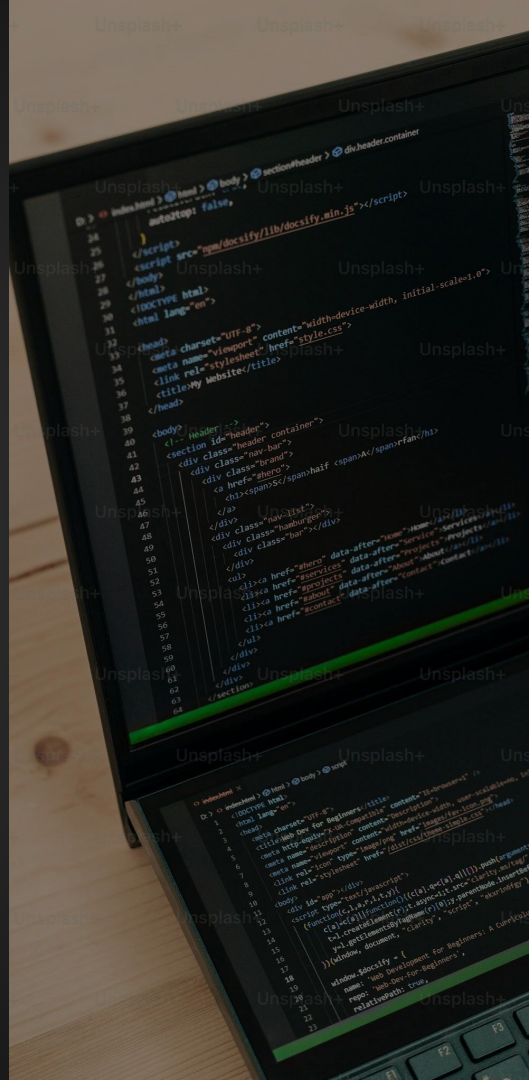
A comunicação assíncrona entre microsserviços traz diversos benefícios em termos de escalabilidade, resiliência e desacoplamento, mas também pode introduzir desafios como complexidade e consistência eventual.



Webhook

Um **Webhook** é um mecanismo de comunicação entre sistemas que permite que um serviço notifique outro sobre eventos em tempo real.

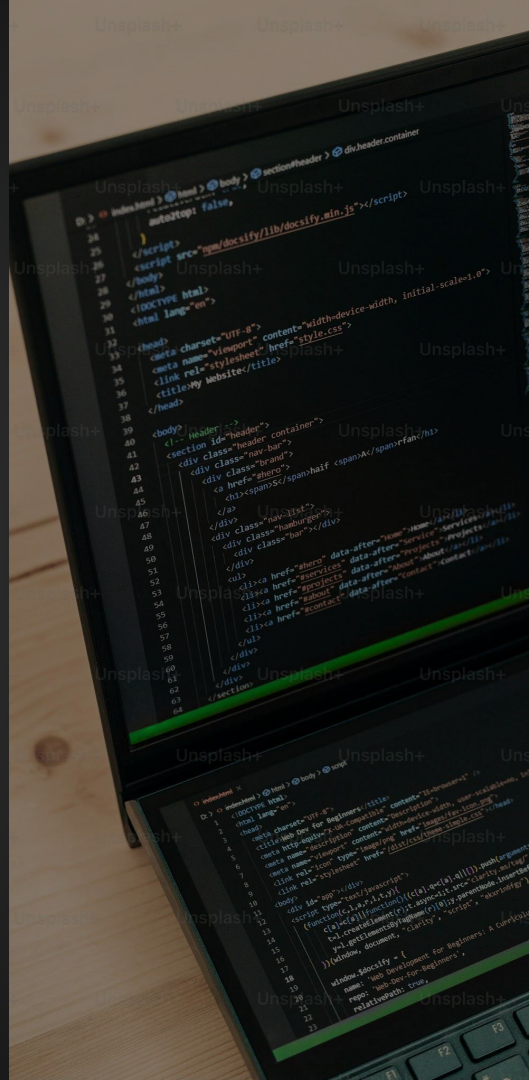
Em vez de um sistema consultar periodicamente outro para verificar se há novas informações, um webhook permite que o sistema de origem envie automaticamente uma notificação para o sistema de destino assim que um evento ocorre.



Webhook

O **Webhook** é considerado uma forma de comunicação assíncrona porque ele não exige uma resposta imediata do sistema receptor para continuar o processamento.

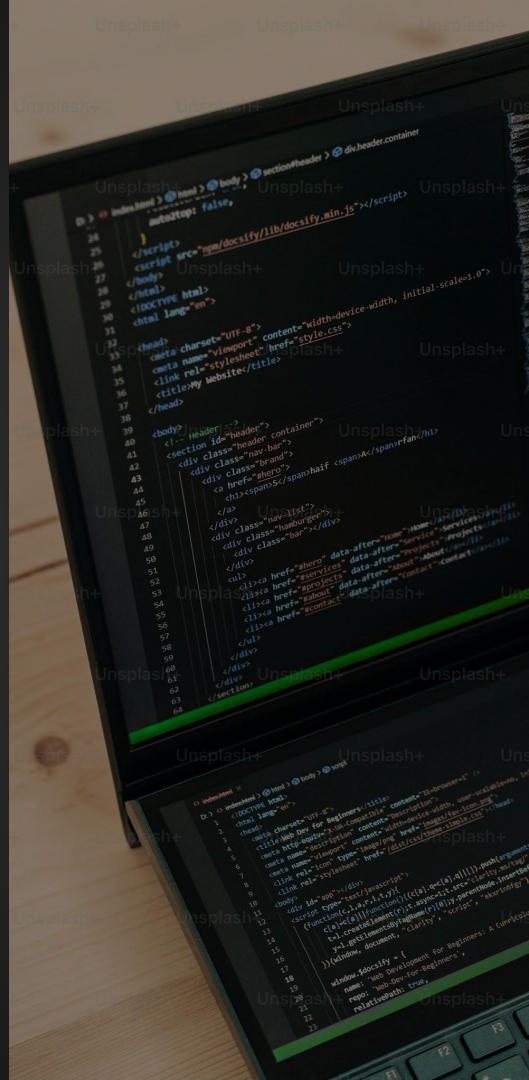
Em vez disso, ele envia um evento e continua sua execução sem esperar que o outro sistema processe a informação no mesmo instante.

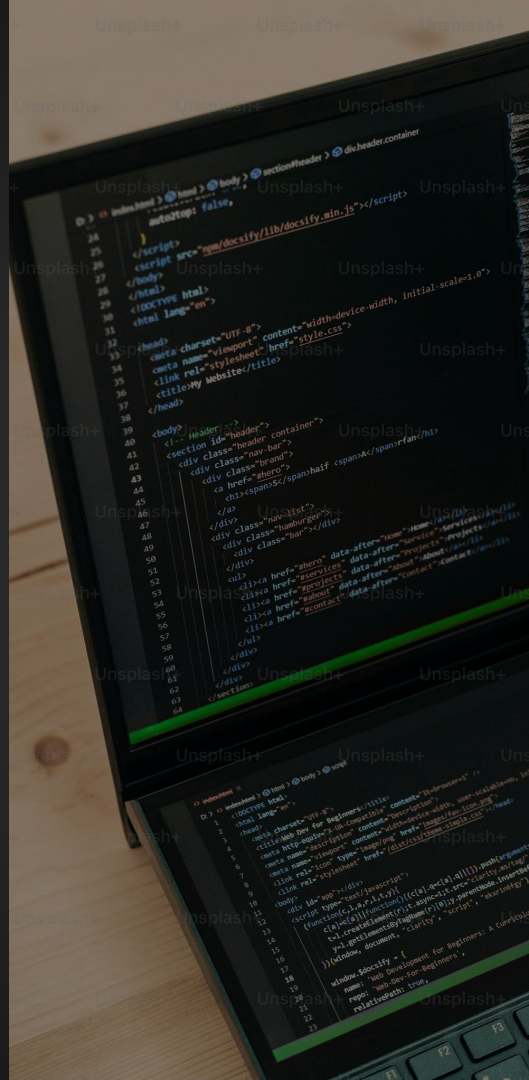


Mensageria

A comunicação assíncrona através de mensageria é um modelo de comunicação entre microsserviços no qual os sistemas se comunicam indiretamente por meio de filas ou tópicos de mensagens, sem precisar de uma resposta imediata.

Essa abordagem é amplamente utilizada para garantir escalabilidade, resiliência e desacoplamento entre serviços.



Mensageria - Características

- **Desacoplamento** - Os serviços não precisam conhecer diretamente uns aos outros.
- **Escalabilidade** - Reduz bloqueios e permite o uso eficiente de recursos.
- **Tolerância a falhas** - Se um serviço estiver temporariamente indisponível, a mensagem pode ser armazenada e processada mais tarde.